

7/24/23, 2:58 PM

Codio

Project

File

Edit

Find

View

Tools

Education

Help

Run

Project index (stand)

Debug

Terminal x

```
codio@athena1ight-nevadalopez:~/workspace$ mkdir example1
codio@athena1ight-nevadalopez:~/workspace$ cd example1
codio@athena1ight-nevadalopez:~/workspace/example1$ python3 -m venv venv
codio@athena1ight-nevadalopez:~/workspace/example1$ source venv/bin/activate
(venv) codio@athena1ight-nevadalopez:~/workspace/example1$ deactivate
codio@athena1ight-nevadalopez:~/workspace/example1$
```

Codio - Third-Party Packages

Guide x

Collapse

Virtual Environments

Installing Third-Party Packages

Before we start installing third-party packages, we need to discuss how Python stores installed packages. By default, Python installs third-party packages for the entire system. That is, if you install `requests` for one project, you can use the package again in another project without having to install it again. This might sound like a good thing, but it can cause problems. Packages often depend on other packages. These are called dependencies.

Let's say you install package `x` which requires the dependency `acme v1.3`. You then later install package `y` which has the dependency `acme v1.1`. Python overwrites `acme v1.3` with the earlier version `acme v1.1`. Now package `x` no longer works because one of its dependencies is broken.

Other potential problems could arise if you mix third-party packages with the standard library. There could be side effects from all of these packages being in the same place. Or, updating the operating system could affect third-party packages stored alongside the standard library.

Virtual Environments

The way to avoid the aforementioned problems is to use a virtual environment for your Python project. The built-in Python virtual environment is called `venv`. Using a virtual environment is similar to having a separate directory with your code, the Python language, and any packages you install. A virtual environment is completely isolated from the larger Python system and other virtual environments. Using `venv` means you avoid the problems of dependency mismatches. It is a good idea to create a virtual environment for every Python project you start.

Before creating a virtual environment, make sure you have a dedicated directory for your project. Your code and the virtual environment should go in this directory. Copy/paste the command in the terminal to make the directory `example1`. Press `ENTER` to run the command.

```
mkdir example1
```

You should now see the `example1` directory in the file tree.

Installing Packages

example1

.settings

Each Python project should have its own virtual environment. So we need to change to the newly created directory with the command below.

```
cd example1
```

Once we are in our project's folder, we can create our virtual environment. The command below creates a virtual environment named `venv`.

```
python3 -m venv venv
```

Python added the `venv` virtual environment to the `example1` directory. To verify this, click on the arrow next to `example1` in the file tree. Do the same thing for `venv`. This is the contents of your virtual environment. Your file tree should look like this:

example1

venv

bin

include

lib

share

lib64

pyvenv.cfg

.settings

Finally, we can activate our virtual environment. This is important to note, virtual environments do not run constantly in the background. You should activate them every time you want to work on your project.

```
source venv/bin/activate
```

We can verify your virtual environment is working by looking at our prompt in the terminal. `(venv)` should appear at the beginning of your prompt. **Note**, your prompt will look different than the one below, but as long as it has `(venv)` you are ready to go.

(venv) codio@schoolbetty-lessongyro:~/workspace/example1\$

Try this variation:

- Delete your virtual environment. In the file tree, open the `example1` directory. Right-click on the `venv` directory and select delete.
- In the terminal, make sure that you are in the `example1` directory. Create a new virtual environment, but this time give it the unique name `example-venv`.

```
python3 -m venv example-venv
```

- Active the new virtual environment.

```
source example-venv/bin/activate
```

- In the file tree, the virtual environment `venv` should be replaced by `example-venv`.

▼ Did you notice?

There was no special command to run from the terminal to uninstall the virtual environment. It is just a directory. Keep in mind, if you install third-party packages and then remove the virtual environment you will have to reinstall them.

deactivate

Reading Question

Select all of the reasons why you should use a virtual environment. **Hint**, there is more than one correct answer.

☐ Virtual environments are required

☐ Virtual environments keep third-party packages separate from the standard library

☐ Virtual environments speed up your program

☐ Virtual environments allow you to have different versions of the same package installed

The correct answers are:

- Virtual environments keep third-party packages separate from the standard library
- Virtual environments allow you to have different versions of the same package installed

Using a virtual environment creates a directory separate from the standard library. Third-party packages are installed here instead of with the standard library. Without virtual environments, Python will only let you have one version of a package installed at any given time. You can install a different version of the same package as long as each one goes in their own virtual environment.

Check It!

Next

https://rodo.com/sandpaw-day/third-party-packages/terminal-backend-guide

1/1